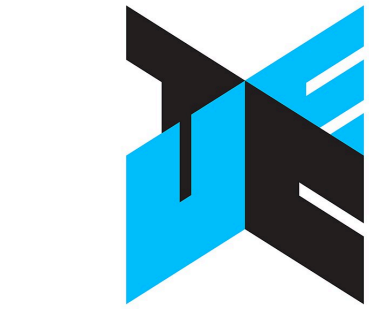


UTEC

UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

INFORME TÉCNICO DE LA PRIMERA ENTREGA

Procesador RISC-V segmentado y arquitectura Dual Issue preliminar

ASIGNATURA: Arquitectura de Computadoras (CS3051)-Teo. 1-Lab. 11-2026-2

DOCENTE: Williams Valencia, Carlos Eduardo

FECHA DE ENTREGA: 02/11/2026

INTEGRANTES:

INTEGRANTES	CÓDIGO
Ladron de Guevara Aguirre, Diego Abraham	202410083
Burgos Ochoa, Jeseph Imanol	202510395
Morales Portella Santiago Cesar	202510403
Heredia Cadenas, Guillermo Arturo	202410219

Lima, Perú

2026

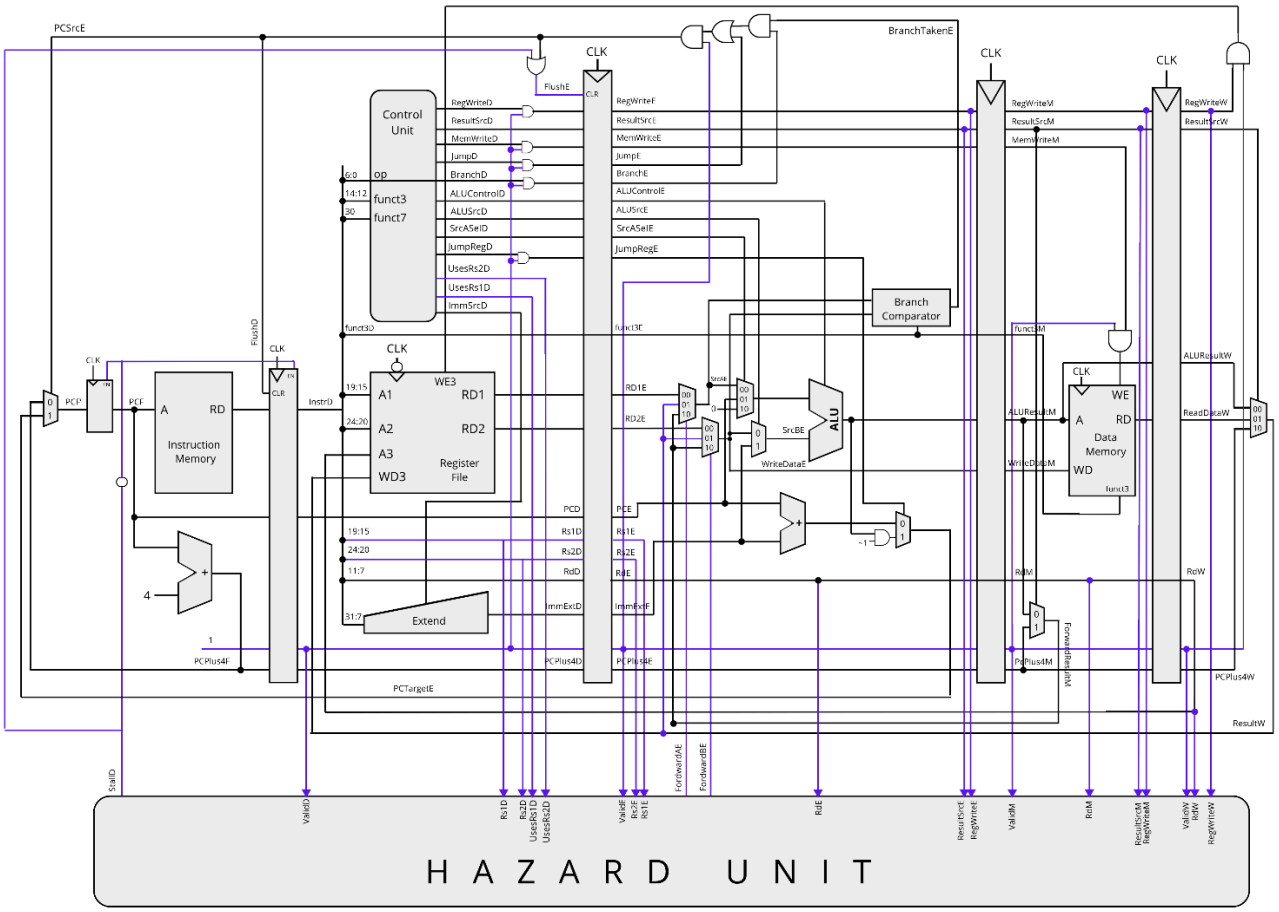
1 Introducción y alcance

La primera entrega (E1) comprende un procesador de una vía para las instrucciones RV32I implementadas, con cinco etapas, control de riesgos y cuatro pruebas funcionales. También incluye el inicio de la arquitectura Dual Issue: lectura y decodificación de dos instrucciones por ciclo, dos vías hasta EX y sus señales de retención y anulación. La ejecución simultánea completa de dos instrucciones aún no es requisito de E1.

2 Pipeline de cinco etapas

La Figura 1 muestra el datapath Single Issue implementado: PC y memoria de instrucciones, Register File, ALU, memoria de datos y selección de writeback, separados por registros entre etapas. También señala las rutas de forwarding y las señales de stall y flush de la Hazard Unit.

Figura 1. Datapath del procesador Single Issue de cinco etapas.



Nota. Elaboración propia. https://canva.link/asg9gb3kk3jpq3e

Un punto importante a tener en cuenta para esta microarquitectura son los procesos que se siguen para cada etapa del pipeline. De manera general, podemos establecer lo siguiente:

Por una parte, para la etapa **Fetch (IF)** el PC entra a la memoria (Instruction Memory) y se carga la instrucción correspondiente en el mismo ciclo. Además, en el mismo ciclo, el sumador calcula el siguiente *PC (PC+4)* el cual pasa por el cable con nombre *PCPlus4F*. También es menester mencionar la lógica que se sigue para elegir qué cargar en el flanco. En este sentido, si *PCSrcE*=1, implica que se toma un salto; por otro lado, si no es 1 y !*StallD*=1 se toma el valor de *PCPlus4F;* y

finalmente, si no hay StallD se queda igual. De manera similar se aplica la lógica para el registro *IF/ID.* En este sentido, si *PCSrcE=1* se carga un NOP con *ValidD=0*, lo cual resulta apropiado porque se está tomando un salto en la instrucción. Caso contrario, si *PCSrcE* no es 1 y *!StallD=1* se carga lo correspondiente a la siguiente instrucción, en su orden natural, y *ValidD=1*. Finalmente, si no hay *StallD*, retiene.

Por otra parte, en la etapa de **Decode (ID)** se separan los campos de la instrucción de acuerdo al rango de bits, de manera que los campos llegan a los diferentes cables, como *Rs1D*, *Rs2D*, *RdD,* etc. Asimismo, en esta etapa, el controller recibe los campos correspondientes al *op*, *funct3* y *funct7* y genera señales de control como *RegWrite, ALUSrc, ResultSrc, ImmSrc, Branch, Jump,* etc. Por otra parte, ciertos campos de la instrucción también ingresan al register file, lo que permite leer *RD1* y *RD2*. Asimismo, tambien llega un campo al bloque extend, el cual según el *ImmSrc* genera *ImmExtD*. Un aspecto importante a mencionar es la incorporación de los cables *UsesRs1D* y *UsesRs2D*, los cuales nos indican si la instrucción realmente lee *rs1* y *rs2*, lo que es de suma importancia para procesar ciertos tipos de instrucción. En lo que respecta al registro *ID/EX*, permite que los cables pasen a su versión *_E* (etapa *Execute*). Asimismo, es menester resaltar que para ciertos cables que salen del controller se realiza un enmascaramiento con *ValidD*. Finalmente, también se precisa que el reset sigue la lógica *PCSrcE* || *StallD*, de manera que si se activa, se carga una burbuja, es decir que todos los valores que pasan a su versión *_E* quedan en cero, incluido *ValidE*.

En otro punto, respecto a la etapa **Execute (EX),** esta corresponde a la parte con mayor contenido lógico combinacional, pues se disponen de diferentes *muxes, ALU*, *sumador* y un bloque *branch Comparator*, siendo este último de suma importancia para determinar si se realizará un salto y así "avisar" a las instrucciones siguientes. La lógica de los valores que se pueden tomar en cada *MUX* se presenta en el diagrama, pero resulta importante resaltar que la forma en cómo está organizado permite abarcar diversos tipos de instrucciones bajo una misma lógica. Como se mencionó anteriormente, en esta etapa también se calcula *PCSrcE* bajo la ecuación

PCSrcE = ValidE && (JumpE || (BranchE && BranchTakenE))

En lo que respecta a la etapa **Memory (MEM)**, se destacapa principalmente porque se realizan las operaciones en memoria (*Data Memory*), de manera que el adress que ingresa corresponde al *ALUResultM*. Asimismo, este bloque utiliza el valor de *funct3M* que es enviado desde la etapa decode. El valor leído de la memoria se almacena en *ReadData*, el cual pasa por el registro (*MEM/WB*) y se queda como *ReadDataW*. Ahondando más en el *data memory*, este escribe en el mismo flanco si *MemWrite=1*, el cual con la ecuación *MemWrite = ValidM && MemwWriteM*

Finalmente, para la etapa **Writeback (WB)** principalmente se tiene un *MUX* con tres entradas que permite "calcular" *resultW*. Asimismo, en esta etapa se hace el cálculo del *RegWrite*, bajo la ecuación *RegWrite = ValidW && RegWriteW*

3 Unidad de control y Hazard Unit

El Controller decodifica la instrucción en ID y genera selección de ALU, acceso a memoria, escritura de registro y tipo de resultado. La Hazard Unit compara los registros fuente de la instrucción en EX con los destinos válidos en MEM y WB. Cuando hay coincidencia, selecciona el dato más reciente: MEM tiene prioridad sobre WB; una carga en MEM espera su dato y no se reenvía como resultado ALU. Si una carga en EX alimenta a la instrucción de ID, StallD retiene PC e IF/ID durante un ciclo e inserta una burbuja en ID/EX. Un branch o salto tomado en EX redirige el PC y anula las instrucciones jóvenes de IF e ID mediante flush.

Un punto importante a mencionar en la interacción con la Hazard Unit es la incorporación de la salida StallD. Como se mencionó anteriormente esta resulta importante en etapas como fetch y decode. Este valor nos permite detener la parte delantera del pipeline cuando una instrucción necesita un dato que todavía no existe; por ejemplo, en una instrucción lw x2, 4(s0) seguida de un add x3,x2,x0 evita usar el valor viejo (antes del lw) del x2.

Por otra parte, también es importante resaltar la incorporación de los bits Valid(*), los cuales permiten saber si en la etapa hay una instrucción real o una burbuja. Por lo mismo, se usa como source para los componentes and en ciertos cables del controller al pasar de etapa, pues limita lo que debe/puede hacer una burbuja. Si bien, para el single issue estos valid actúan más como una “segunda capa de protección”, estos tendrán mayor importancia en el dual-issue.

4 Verificación funcional

Se ejecutó ./run_e1.ps1 para ensamblar los programas .mem, compilar con Icarus Verilog y correr los testbenches. Cada testbench produjo un archivo VCD con las señales internas. Las figuras de esta sección son capturas directas de esos VCD abiertos en GTKWave; las tablas reúnen valores leídos en las ondas y resultados finales comprobados por los testbenches.

Programa	Qué comprueba
Programa 1	ISA soportada sin dependencias que exijan Hazard Unit
Programa 2	Forwarding entre etapas y hacia un store
Programa 3	Load Use: retención y burbuja
Programa 4	BEQ y JAL: redirección y anulación

4.1 Programa 1 ISA sin dependencias

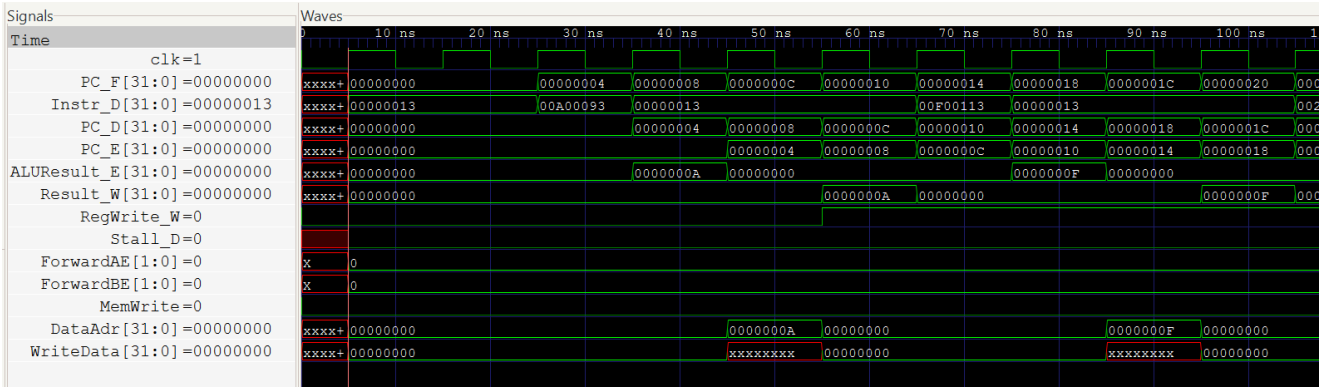
El Programa 1 usa el ensamblador mostrado a continuación. Incluye ADDI, ADD, SUB, SLT, AND, OR, SLL, SRL, SLTI, ANDI, ORI, SLLI, SRLI, LW y SW; las dos capturas permiten ubicar cada instrucción por su PC.

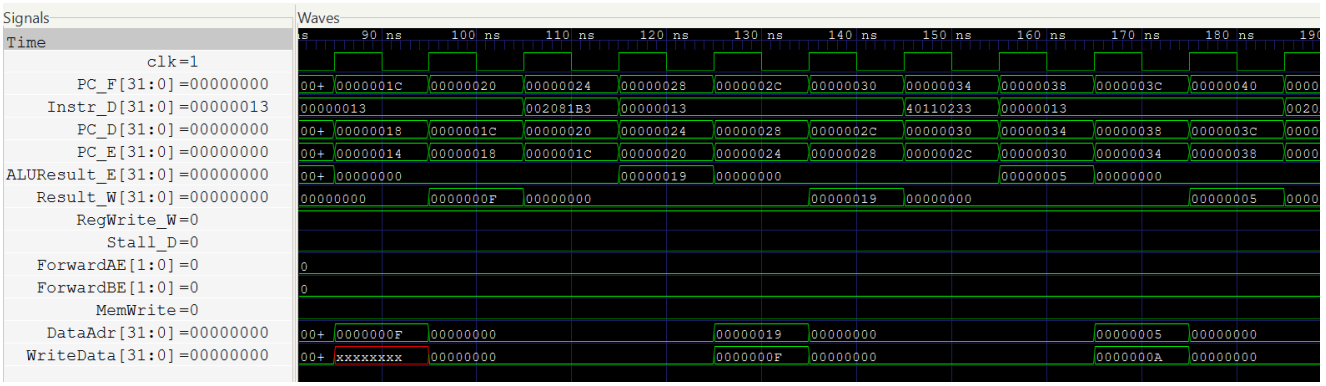
```
addi x1, x0, 10                 # PC=000
nop                             # PC=004
nop                             # PC=008
nop                             # PC=012
addi x2, x0, 15                 # PC=016
nop                             # PC=020
nop                             # PC=024
nop                             # PC=028
add x3, x1, x2                  # PC=032
nop                             # PC=036
nop                             # PC=040
nop                             # PC=044
sub x4, x2, x1                  # PC=048
nop                             # PC=052
nop                             # PC=056
nop                             # PC=060
slt x5, x1, x2                  # PC=064
nop                             # PC=068
nop                             # PC=072
nop                             # PC=076
and x6, x1, x2                  # PC=080
nop                             # PC=084
nop                             # PC=088
nop                             # PC=092
or x7, x1, x2                   # PC=096
nop                             # PC=100
nop                             # PC=104
nop                             # PC=108
sll x8, x5, x1                  # PC=112
nop                             # PC=116
nop                             # PC=120
nop                             # PC=124
srl x9, x8, x5                  # PC=128
nop                             # PC=132
nop                             # PC=136
nop                             # PC=140
slti x12, x1, 11                # PC=144
nop                             # PC=148
nop                             # PC=152
```

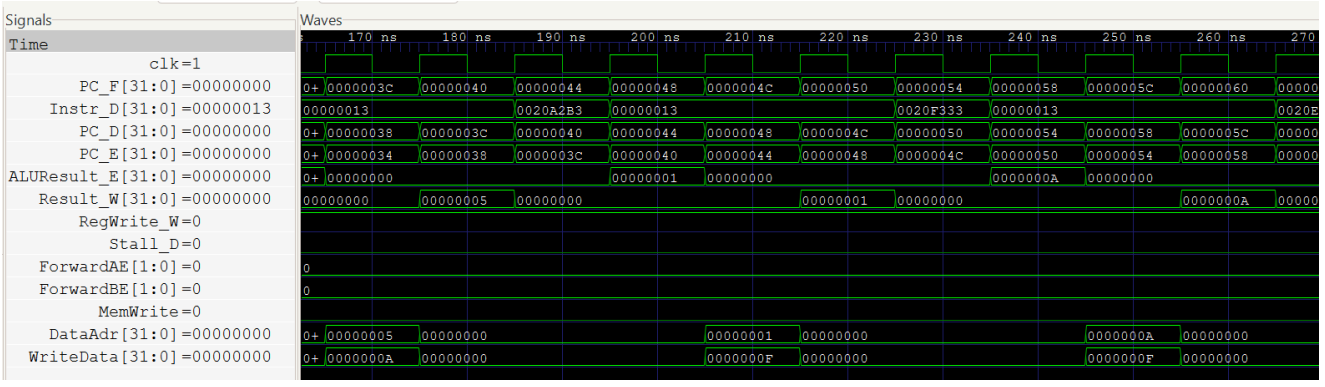
```
40  nop                         # PC=156
41  andi x13, x2, 7             # PC=160
42  nop                         # PC=164
43  nop                         # PC=168
44  nop                         # PC=172
45  ori x14, x1, 1              # PC=176
46  nop                         # PC=180
47  nop                         # PC=184
48  nop                         # PC=188
49  slli x15, x12, 2            # PC=192
50  nop                         # PC=196
51  nop                         # PC=200
52  nop                         # PC=204
53  srli x16, x15, 1            # PC=208
54  nop                         # PC=212
55  nop                         # PC=216
56  nop                         # PC=220
57  sw x1, 96(x0)               # PC=224
58  nop                         # PC=228
59  nop                         # PC=232
60  nop                         # PC=236
61  lw x10, 96(x0)              # PC=240
62  nop                         # PC=244
63  nop                         # PC=248
64  nop                         # PC=252
65  add x11, x10, x2            # PC=256
66  nop                         # PC=260
67  nop                         # PC=264
68  nop                         # PC=268
69  sw x11, 100(x0)             # PC=272
70  nop                         # PC=276
71  nop                         # PC=280
72  nop                         # PC=284
```

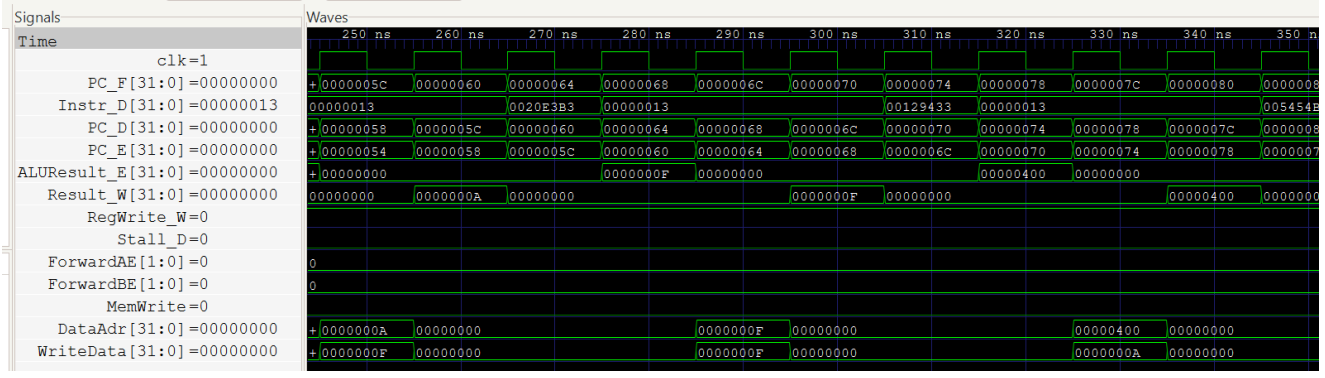
Entre un productor y su consumidor se colocaron tres NOP. Por ello la prueba recorre las instrucciones soportadas sin necesitar forwarding ni stalling para obtener el resultado correcto. Los NOP también explican que la traza ocupe 72 ciclos.

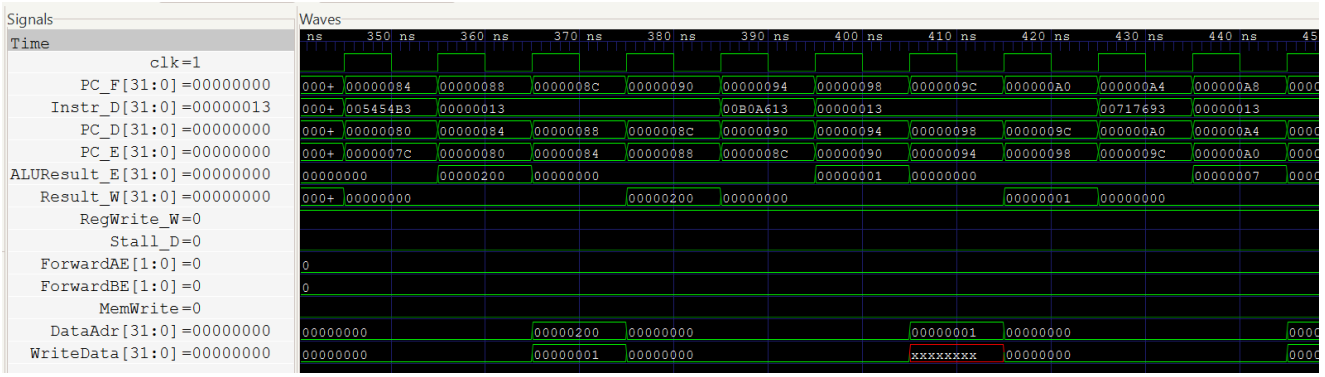
Las nueve capturas siguientes recorren la simulación desde el inicio hasta el store final. Se observan el avance del PC, la operación de la ALU, el dato que llega a WB y las escrituras de memoria. La tabla resume valores finales que el testbench compara con los esperados.

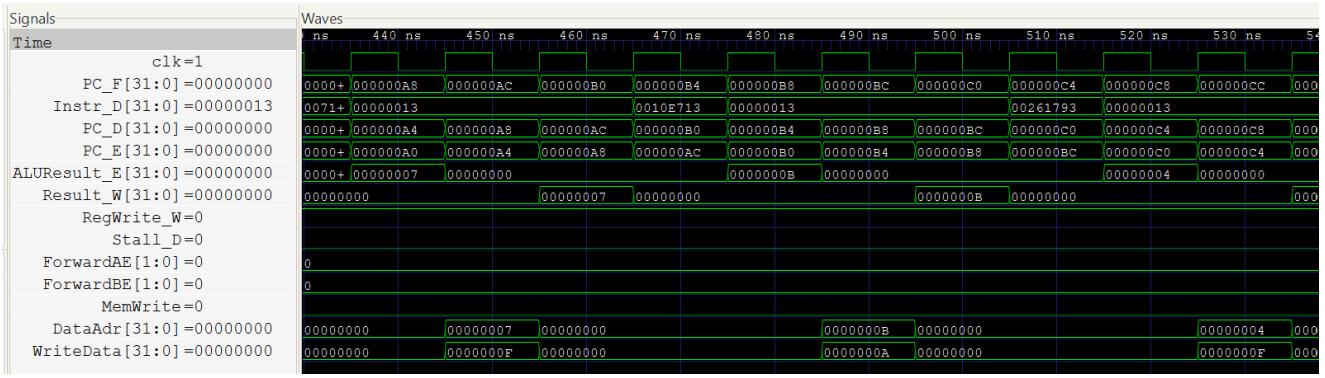

Signals
Waves
Time
clk=1
PC_F[31:0]=00000000
Instr_D[31:0]=00000013
PC_D[31:0]=00000000
PC_E[31:0]=00000000
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000000
RegWrite_W=0
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=0
DataAdr[31:0]=00000000
WriteData[31:0]=00000000

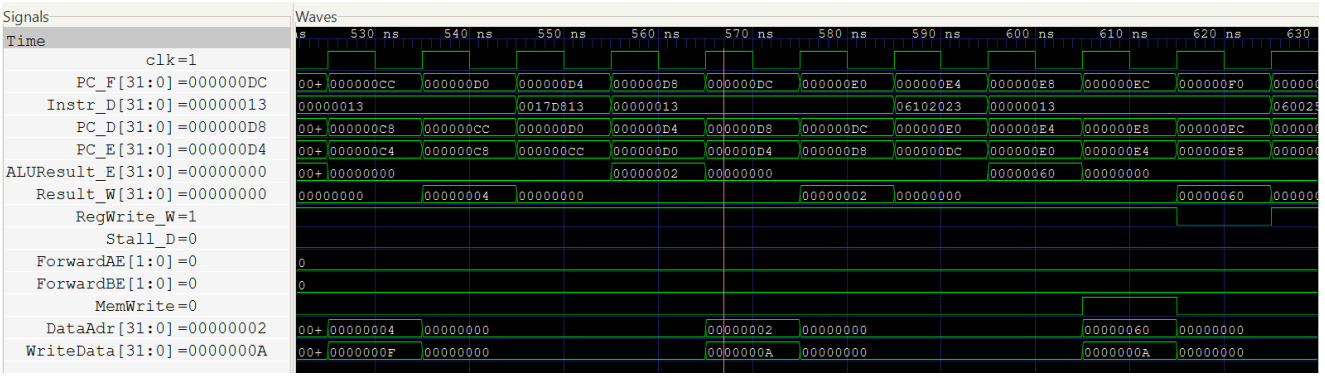

Signals
Waves
Time
clk=1
PC_F[31:0]=00000000
Instr_D[31:0]=00000013
PC_D[31:0]=00000000
PC_E[31:0]=00000000
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000000
RegWrite_W=0
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=0
DataAdr[31:0]=00000000
WriteData[31:0]=00000000

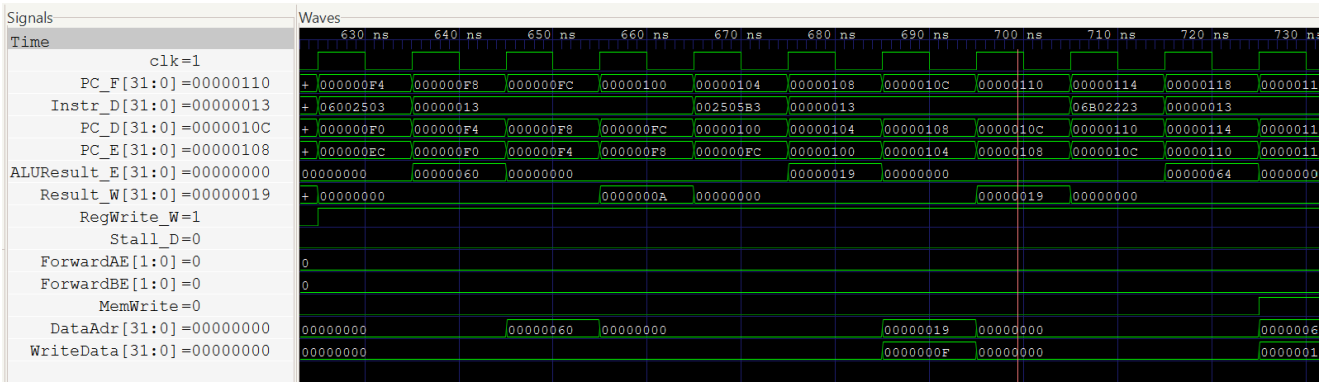

Signals
Waves
Time
clk=1
PC_F[31:0]=00000000
Instr_D[31:0]=00000013
PC_D[31:0]=00000000
PC_E[31:0]=00000000
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000000
RegWrite_W=0
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=0
DataAdr[31:0]=00000000
WriteData[31:0]=00000000


Signals
Waves
Time
clk=1
PC_F[31:0]=00000000
Instr_D[31:0]=00000013
PC_D[31:0]=00000000
PC_E[31:0]=00000000
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000000
RegWrite_W=0
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=0
DataAdr[31:0]=00000000
WriteData[31:0]=00000000


Signals
Time
clk=1
PC_F[31:0]=00000000
Instr_D[31:0]=00000013
PC_D[31:0]=00000000
PC_E[31:0]=00000000
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000000
RegWrite_W=0
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=0
DataAdr[31:0]=00000000
WriteData[31:0]=00000000
Waves
350 ns
360 ns
370 ns
380 ns
390 ns
400 ns
410 ns
420 ns
430 ns
440 ns


Signals
Time
clk=1
PC_F[31:0]=00000000
Instr_D[31:0]=00000013
PC_D[31:0]=00000000
PC_E[31:0]=00000000
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000000
RegWrite_W=0
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=0
DataAdr[31:0]=00000000
WriteData[31:0]=00000000
Waves
440 ns
450 ns
460 ns
470 ns
480 ns
490 ns
500 ns
510 ns
520 ns
530 ns


Signals
Time
clk=1
PC_F[31:0]=000000DC
Instr_D[31:0]=00000013
PC_D[31:0]=000000D8
PC_E[31:0]=000000D4
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000000
RegWrite_W=1
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=0
DataAdr[31:0]=00000002
WriteData[31:0]=0000000A
Waves
530 ns
540 ns
550 ns
560 ns
570 ns
580 ns
590 ns
600 ns
610 ns
620 ns
630


Signals
Time
clk=1
PC_F[31:0]=00000110
Instr_D[31:0]=00000013
PC_D[31:0]=0000010C
PC_E[31:0]=00000108
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000019
RegWrite_W=1
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=0
DataAdr[31:0]=00000000
WriteData[31:0]=00000000
Waves
630 ns
640 ns
650 ns
660 ns
670 ns
680 ns
690 ns
700 ns
710 ns
720 ns
730 ns

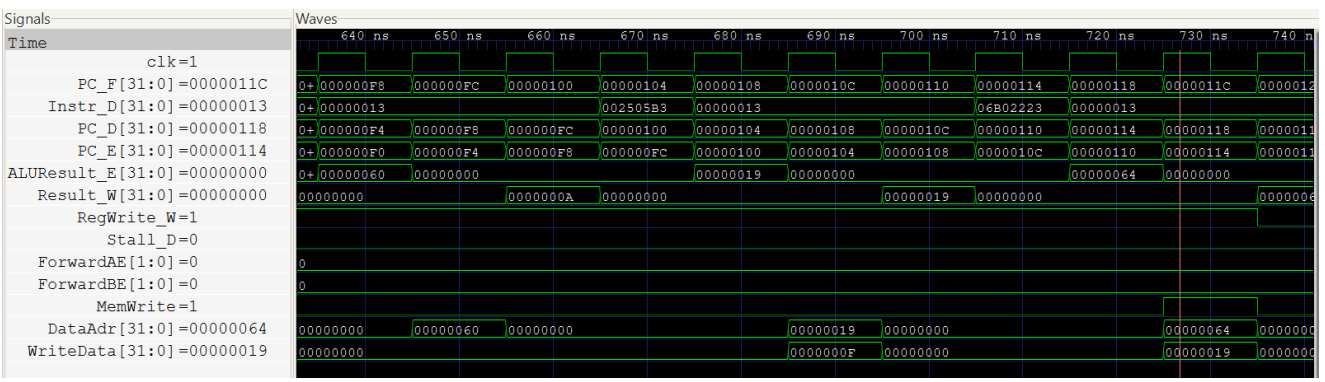

Signals
Time
clk=1
PC_F[31:0]=0000011C
Instr_D[31:0]=00000013
PC_D[31:0]=00000118
PC_E[31:0]=00000114
ALUResult_E[31:0]=00000000
Result_W[31:0]=00000000
RegWrite_W=1
Stall_D=0
ForwardAE[1:0]=0
ForwardBE[1:0]=0
MemWrite=1
DataAdr[31:0]=00000064
WriteData[31:0]=00000019
Waves
640 ns
650 ns
660 ns
670 ns
680 ns
690 ns
700 ns
710 ns
720 ns
730 ns
740 n

Figura 2. Secuencia completa de P1 en GTKWave, obtenida de tests/e1_1.vcd.

Resultado comprobado	Esperado	Observado
x3 = x1 + x2	25	25
x4 = x2 - x1; x5 = (x1 < x2)	5; 1	5; 1
x8 = x5 << x1; x9 = x8 >> x5	1024; 512	1024; 512
mem[96]; x10; x11; mem[100]	10; 10; 25; 25	10; 10; 25; 25

En las ondas, x1=10 y x2=15 llegan a WB antes del ADD que obtiene 25; luego se ven la resta, las comparaciones y los desplazamientos. Cerca de 605 ns aparece el primer store: dirección 96, dato 10. La carga posterior devuelve 10 y el ADD produce x11=25. Cerca de 725 ns se activa el store final en dirección 100 con dato 25. Stall_D permanece en cero durante el programa; los resultados finales de la tabla coinciden con las aserciones del testbench.

4.2 Programa 2 Forwarding

```
addi x1, x0, 10                # PC=000
addi x2, x1, 5                 # PC=004
add x3, x2, x1                 # PC=008
sw x3, 96(x0)                  # PC=012
addi x4, x0, 1                 # PC=016
addi x4, x4, 2                 # PC=020
addi x5, x4, 22                # PC=024
sw x5, 100(x0)                 # PC=028
```

El Programa 2 coloca consumidores inmediatamente después de sus productores. La Hazard Unit debe escoger datos de MEM o WB antes de que lleguen al Register File. Las dos capturas muestran ForwardAE, ForwardBE, operandos en EX, resultado ALU y el dato reenviado hacia el store.



Figura 3. Selección de forwarding y resultados de P2 en GTKWave, desde tests/e1_2.vcd.

t (ns)	Instrucción en EX	Forward A/B	Operandos usados	Resultado ALU
45	ADDI x2, x1, 5	2/0	10 + 5	15
55	ADD x3, x2, x1	2/1	15 + 10	25
85	ADDI x4, x4, 2	2/0	1 + 2	3
105	SW x5, 100(x0)	0/2	base 0; dato 25	dirección 100

A 55 ns, el ADD recibe x2=15 por la ruta de MEM y x1=10 por WB; la ALU entrega 25 sin detener el pipeline. A 85 ns, el ADDI usa x4=1 reenviado y el inmediato 2 para producir 3: el valor de ForwardedB_E se ve en la onda, pero esa instrucción no lo usa como segundo operando ALU. A 105 ns, ForwardBE=2 aporta 25 como dato del store; en MEM se escribe 25 en la dirección 100. Stall_D permanece en cero.

4.3 Programa 3 Stalling

```
addi x1, x0, 25                 # PC=000
sw x1, 96(x0)                   # PC=004
lw x2, 96(x0)                   # PC=008
add x3, x2, x0                  # PC=012
sw x3, 100(x0)                  # PC=016
```

El Programa 3 almacena 25 en la dirección 96, lo carga con LW a x2 y usa x2 de inmediato. Ese dato aún no está disponible cuando el consumidor entra en ID. La Hazard Unit detiene PC e IF/ID un ciclo, inserta una burbuja en EX y permite que la instrucción continúe cuando el valor cargado pueda reenviarse.

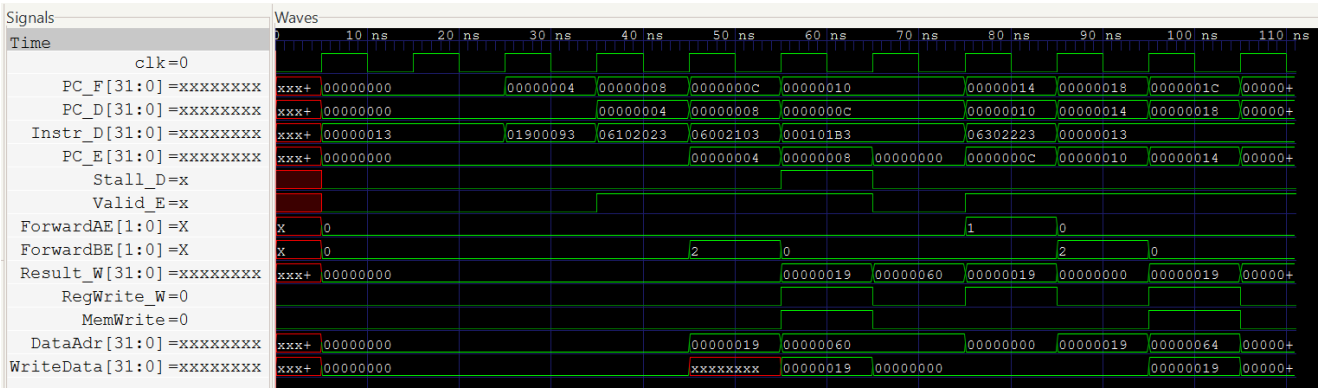


Figura 4. Retención del PC y burbuja Load Use en P3, desde tests/e1_3.vcd.

t (ns)	PC en IF	Stall_D	Valid_E	Resultado WB / escritura
55	16	1	1	25; RegWrite_W=1
65	16	0	0	96; RegWrite_W=0
75	20	0	1	25; RegWrite_W=1

A 55 ns, Stall_D = 1 y el PC queda en 16. A 65 ns, Valid_E=0 identifica la burbuja; aunque el bus Result_W muestra 96, RegWrite_W=0 y ese valor no se escribe en ningún registro. A 75 ns el consumidor vuelve a avanzar y recibe el 25 cargado. La escritura final en memoria[100]=25 confirma que la dependencia se resolvió con un solo ciclo de espera.

4.4 Programa 4 Flushing

```
addi x1, x0, 1                  # PC=000
beq x1, x1, 12                  # PC=004
sw x1, 104(x0)                  # PC=008
addi x2, x0, 99                 # PC=012
addi x2, x0, 25                 # PC=016
jal x3, 12                      # PC=020
sw x1, 104(x0)                  # PC=024
addi x2, x0, 99                 # PC=028
beq x1, x0, 8                   # PC=032
sw x2, 100(x0)                  # PC=036
```

El Programa 4 prueba cambios del flujo de control. El BEQ de PC=4 se toma hacia PC=16; el JAL de PC=20 salta a PC=32 y deja PC+4=24 como enlace en x3. Un BEQ posterior no se toma. Las ondas permiten comprobar tanto los destinos como la anulación de instrucciones que habían entrado por el camino incorrecto.

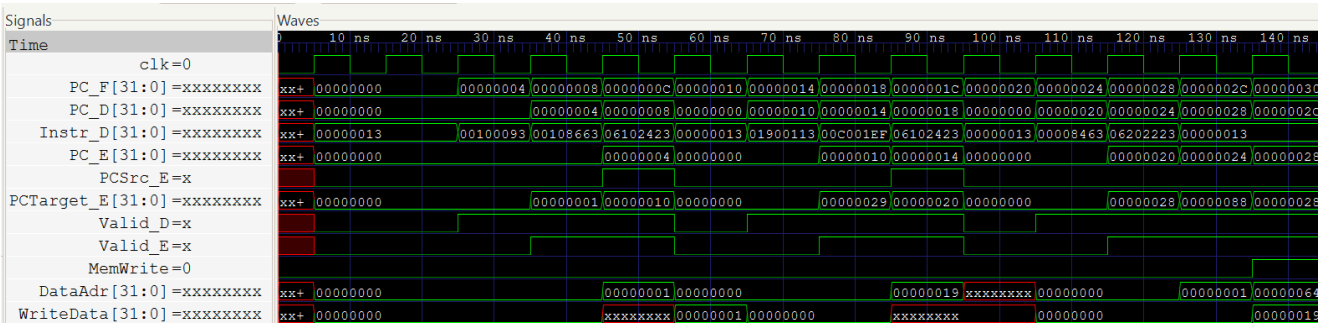


Figura 5. Redirección del PC y anulación de instrucciones en P4, desde tests/e1_4.vcd.

t (ns)	PC en IF	PCSrc_E	Destino tomado	Valid_D / Valid_E
45	12	1	16	1 / 1
55	16	0	-	0 / 0
85	28	1	32	1 / 1
95	32	0	-	0 / 0

Tras el BEQ tomado, el PC pasa a 16 y Valid_D/Valid_E bajan a cero; el mismo patrón se repite después del JAL hacia 32. Esas instrucciones anuladas no activan MemWrite. El testbench

comprueba que los stores de las rutas incorrectas no escribieron en dirección 104. En cambio, el store válido escribió 25 en dirección 100 y x3 conservó el enlace 24.

5 Arquitectura Dual Issue preliminar

El prototipo Dual Issue, amplía Fetch y Decode de una a dos vías y lleva ambos pares de datos y controles hasta EX. Dos ALU permiten comprobar el datapath preliminar. Aún no se implemento el Issue Unit, acceso compartido a memoria, resolución de branches de ambas vías ni doble writeback funcional.

5.1 Cambios de Fetch

En el procesador Single Issue, Fetch lee una instrucción en PC y avanza a PC+4. En rtl/dualissue_prelim.v, PC0_F=PC y PC1_F=PC+4 se envían a los dos puertos de lectura de rtl/imem.v; así se obtiene un par de instrucciones en el mismo ciclo. El par y sus PC quedan registrados para Decode. Si no hay hold, el siguiente PC avanza ocho bytes. hold conserva el par y el PC; flush carga redirect_pc e invalida ambas vías.

5.2 Cambios de Decode y módulos incorporados

Decode usa dos instancias de rtl/controller.v y dos de rtl/extend.v, una por instrucción, para obtener señales de control e inmediatos independientes. rtl/regfile_dual.v aporta cuatro lecturas simultáneas: rs1 y rs2 de cada vía. rtl/dualissue_prelim.v registra por separado los operandos, inmediatos, controles y bits de validez antes de alimentar dos instancias de rtl/alu.v en EX. El Register File admite dos puertos de escritura y tiene una prueba independiente, pero en este prototipo we0 y we1 están conectados a cero; por eso las dos vías todavía no realizan writeback.

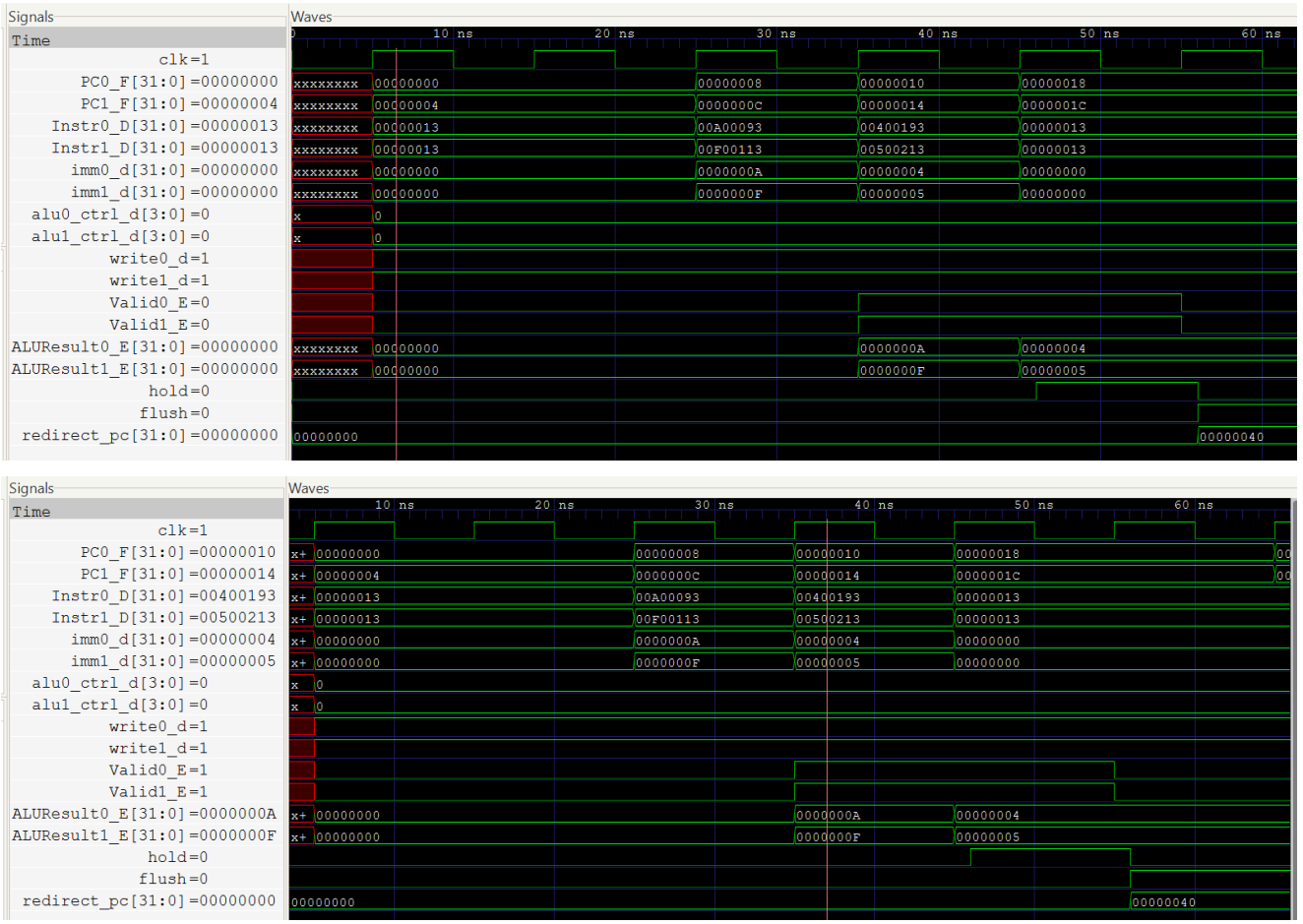


Figura 7. Dual Fetch, Dual Decode, validez y resultados EX en GTKWave, desde tests/e1_dual.vcd.

t (ns)	Instrucciones 0 / 1 en ID	Inmediatos 0 / 1	Valid EX; ALU 0 / 1
25	00A00093 / 00F00113	10 / 15	0; sin resultado válido
35	00400193 / 00500213	4 / 5	1; 10 / 15
45	NOP / NOP	-	1; 4 / 5

A 25 ns se observan simultáneamente en ID las instrucciones de PC=0 y PC=4, con inmediatos 10 y 15. A 35 ns ese par produce 10/15 en las dos ALU, mientras ID ya contiene el par de PC=8 y PC=12, con inmediatos 4/5; a 45 ns se obtienen 4/5. El testbench también comprueba hold: a 55 ns el PC permanece en 24 y Valid_E baja a cero. Con flush y redirect_pc=64, a 65 ns las posiciones de fetch pasan a 64 y 68 y EX queda inválida. La prueba separada del Register File comprueba sus puertos de lectura y escritura, aunque en el prototipo las escrituras se mantienen deshabilitadas.

6 Reproducción y archivos de entrega

Repositorio de Github: https://github.com/DiegoLadrondeGuevara/pipeline.git

Para reproducir la verificación en Windows, abrir una terminal en la raíz del repositorio y ejecutar ./run_e1.ps1 con Python e Icarus Verilog disponibles. El script prepara los .mem, compila los módulos de rtl/ y los testbenches, ejecuta las pruebas y genera los VCD en tests/. Esos VCD se pueden abrir en GTKWave para inspeccionar las mismas señales mostradas en el informe.

La entrega reúne rtl/ (diseño), testbenches/ (aserciones y generación de VCD), tests/ (programas .S, .mem y trazas), run_e1.ps1. Los archivos de tmp/ y los binarios temporales de simulación no forman parte de la entrega.